SYSTEM DESIGN DOCUMENT

Unified Service Scheduler

Technology: C# • ASP.NET Core Web API • EF Core • SQLite • xUnit

1. Executive Summary

The Unified Service Scheduler replaces manual dealership service booking with a REST API that accepts an appointment request, validates the customer, vehicle, dealership and service, checks that a suitably skilled technician and an active service bay are available for the entire service interval, and persists a confirmed appointment. The API is implemented as a modular monolith: a single ASP.NET Core process with controller, booking-service, persistence and domain layers. SQLite is used as a lightweight persistent database suitable for a coding assessment and local demonstration.

The design prioritizes clear business rules, testability and a small operational footprint. It deliberately avoids introducing distributed infrastructure that is not needed for the assessment. The main production concern is concurrency: the current static SemaphoreSlim serializes booking attempts only inside one API process. It does not protect multiple processes or instances. A production deployment would need database-enforced conflict protection or a transactional reservation strategy.

2. Problem Statement and Scope

A customer requests a service appointment for a particular vehicle, service type, dealership and start time. Each service type has a duration and required technician skill. The scheduler must only confirm a booking when both a qualified technician and a service bay can be assigned for the whole interval.

2.1 In scope

- Create a confirmed appointment through POST /api/Appointments.
- Validate required identifiers, future start time, customer/vehicle association, dealership existence and service/dealership association.
- Calculate end time using the service duration.
- Find a technician with the required skill and no overlapping confirmed appointment.
- Find an active service bay with no overlapping confirmed appointment.
- Persist the appointment and return the assigned resource IDs.
- Retrieve an appointment by ID through GET /api/Appointments/{id}.
- Return meaningful HTTP statuses for success, validation failure, missing records and resource conflicts.
- Automated tests for domain overlap rules and booking-service behavior.

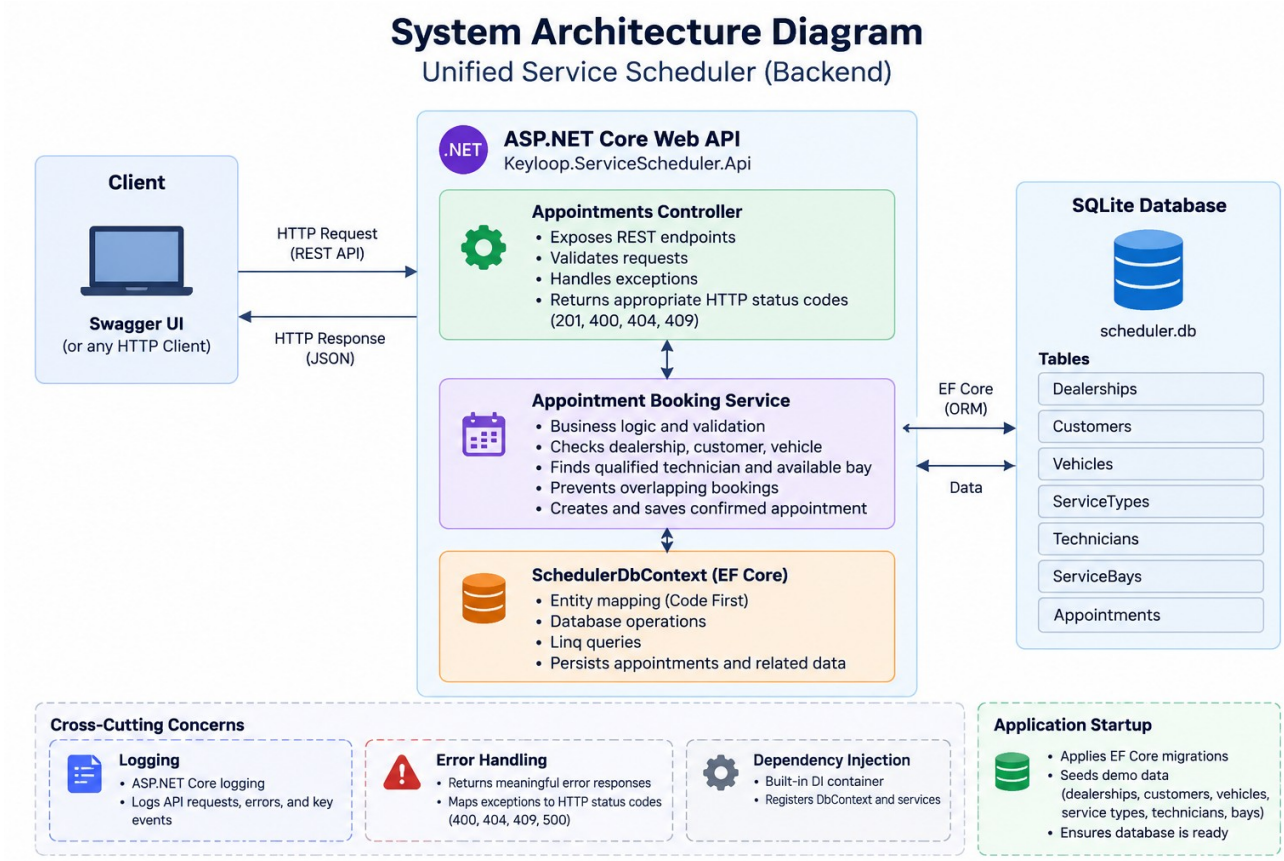
2.2 Out of scope / assumptions

- No authentication/authorization or customer-facing UI is implemented for this assessment; Swagger acts as the API test client.
- No cancellation, rescheduling, recurring booking, payment, notifications, or waitlist is included.
- Appointments are treated as confirmed when persisted successfully. A separate pending/hold state is not implemented.
- The scheduler uses one dealership's resources per request; a service type must belong to the selected dealership.
- A technician qualifies when their Skills collection contains the service's RequiredSkill, compared case-insensitively.
- Appointment intervals use half-open semantics [start, end): an appointment ending at 11:00 does not conflict with one starting at 11:00.
- Only Confirmed appointments reserve technician and bay capacity. Other statuses, if added later, must have explicitly defined reservation semantics.
- The current prototype validates that the vehicle belongs to the supplied customer; additional business rules such as preventing the same vehicle from having overlapping appointments should be added if required by product policy.

3. Architecture Overview

The solution is organized as a modular monolith rather than microservices. A single API deployment keeps the assessment simple to run, debug and test while retaining separation of responsibilities. The API layer handles HTTP concerns; the booking service owns the use-case and business validation; EF Core maps entities and queries the database; SQLite persists the records.

3.1 System architecture diagram



3.2 Component responsibilities

Component	Responsibility	Design rationale
Swagger UI / HTTP client	Submits requests and inspects JSON responses.	Provides a lightweight test harness without requiring a frontend.
AppointmentsController	Defines REST routes, invokes the booking use case, maps outcomes to HTTP responses.	Keeps transport concerns out of business logic.
AppointmentBookingService	Validates booking inputs, calculates end time, evaluates skills and availability, creates the appointment.	Centralizes the booking use case so it can be unit/integration tested.
SchedulerDbContext (EF Core)	Maps entities, exposes DbSets and executes database operations.	Provides typed queries and persistence abstraction.
SQLite	Persists dealership, customer, vehicle, service, technician, bay and appointment records.	Simple local setup; sufficient for a single-instance assessment.
DbSeeder / startup setup	Creates demo reference data and prepares the database if configured at application startup.	Makes the application demonstrable without manual data entry.
ASP.NET Core logging and error mapping	Captures framework/application diagnostics and maps known failures to HTTP responses.	Helps diagnose failures; structured business metrics/tracing can be added for production.

4. Booking Sequence and Data Flow

4.1 Sequence diagram

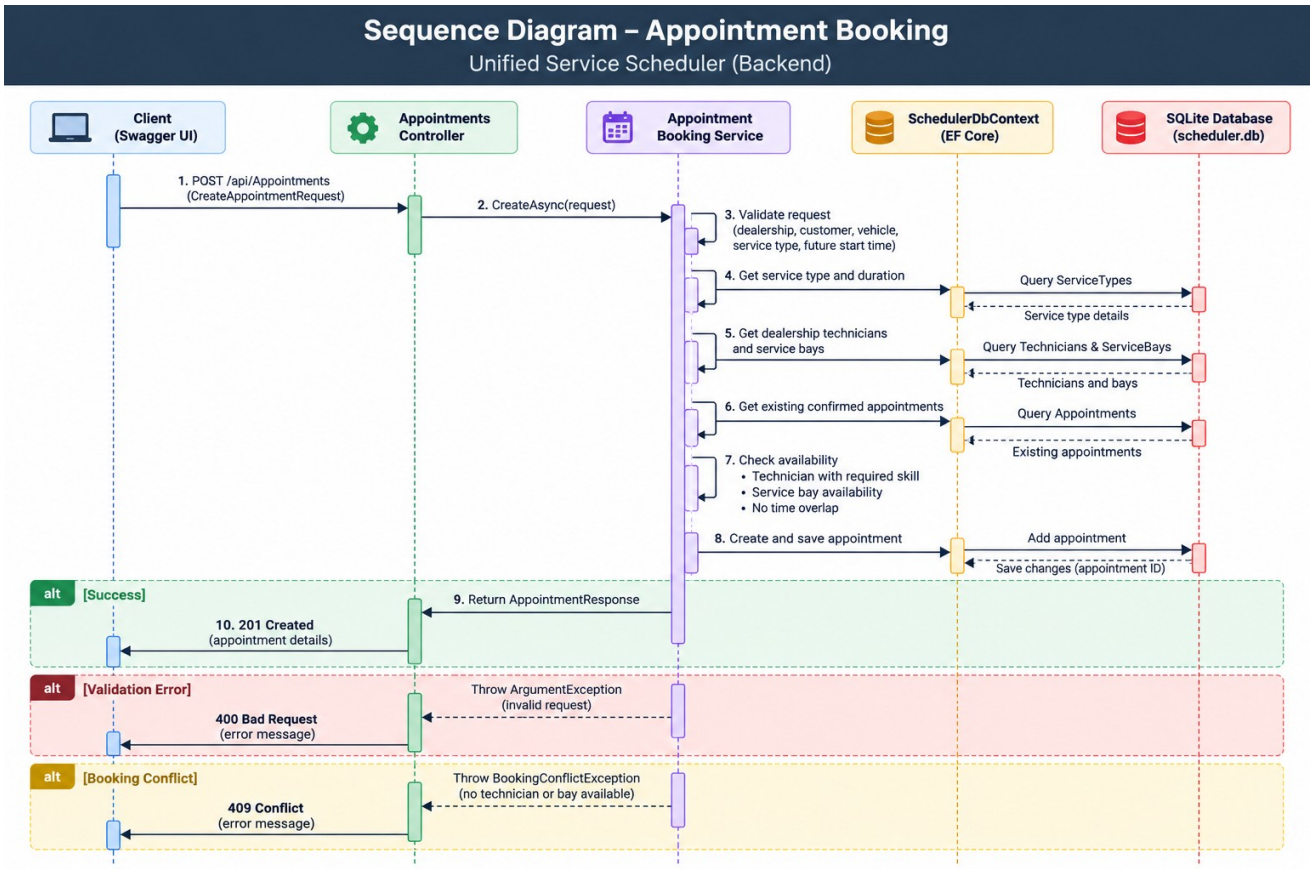


Figure 2. Request/response sequence for successful and rejected booking attempts.

4.2 Successful request flow

1. Client sends POST /api/Appointments with dealershipId, customerId, vehicleId, serviceTypeId and startTime.
2. ASP.NET Core model binding and request validation run; the controller calls AppointmentBookingService.CreateAsync.
3. The service rejects missing IDs or a default/past start time and normalizes the requested start time to UTC.
4. The service confirms the dealership and customer exist, and that the vehicle belongs to that customer.
5. The service loads the service type for the selected dealership and reads its duration and required skill.
6. The service calculates endTime = startTime + duration.
7. The service loads technicians and active service bays for the dealership, plus confirmed appointments used for overlap comparisons.
8. A technician is eligible when they have the required skill and no overlapping confirmed appointment. A bay is eligible when active and not occupied by an overlapping confirmed appointment.
9. The service creates an Appointment with the selected technician and bay and saves it using SaveChangesAsync.

10. The controller returns 201 Created with the response body and a Location header pointing to the appointment resource.

4.3 Time-overlap rule

Two appointment intervals overlap when:

```
requestedStart < existingEnd && requestedEnd > existingStart
```

This is the standard half-open interval rule. It rejects partial overlaps, containment, and identical intervals, while allowing adjacent bookings (for example, 10:00–11:00 followed by 11:00–11:30). Both technician and bay availability use the service's complete duration, not just its start time.

4.4 Failure paths

Condition	HTTP status	Expected behavior
Missing/invalid IDs, unknown dealership/customer/vehicle/service, past start time	400 Bad Request	Return an error message that identifies the validation problem.
No qualified technician available	409 Conflict	Do not persist a confirmed appointment.
No active service bay available	409 Conflict	Do not persist a confirmed appointment.
Appointment ID does not exist	404 Not Found	Return a not-found response.
Unexpected persistence/provider failure	500 Internal Server Error	Return a generic server error and log details server-side; do not expose internal exception details.

5. Booking Availability Flowchart

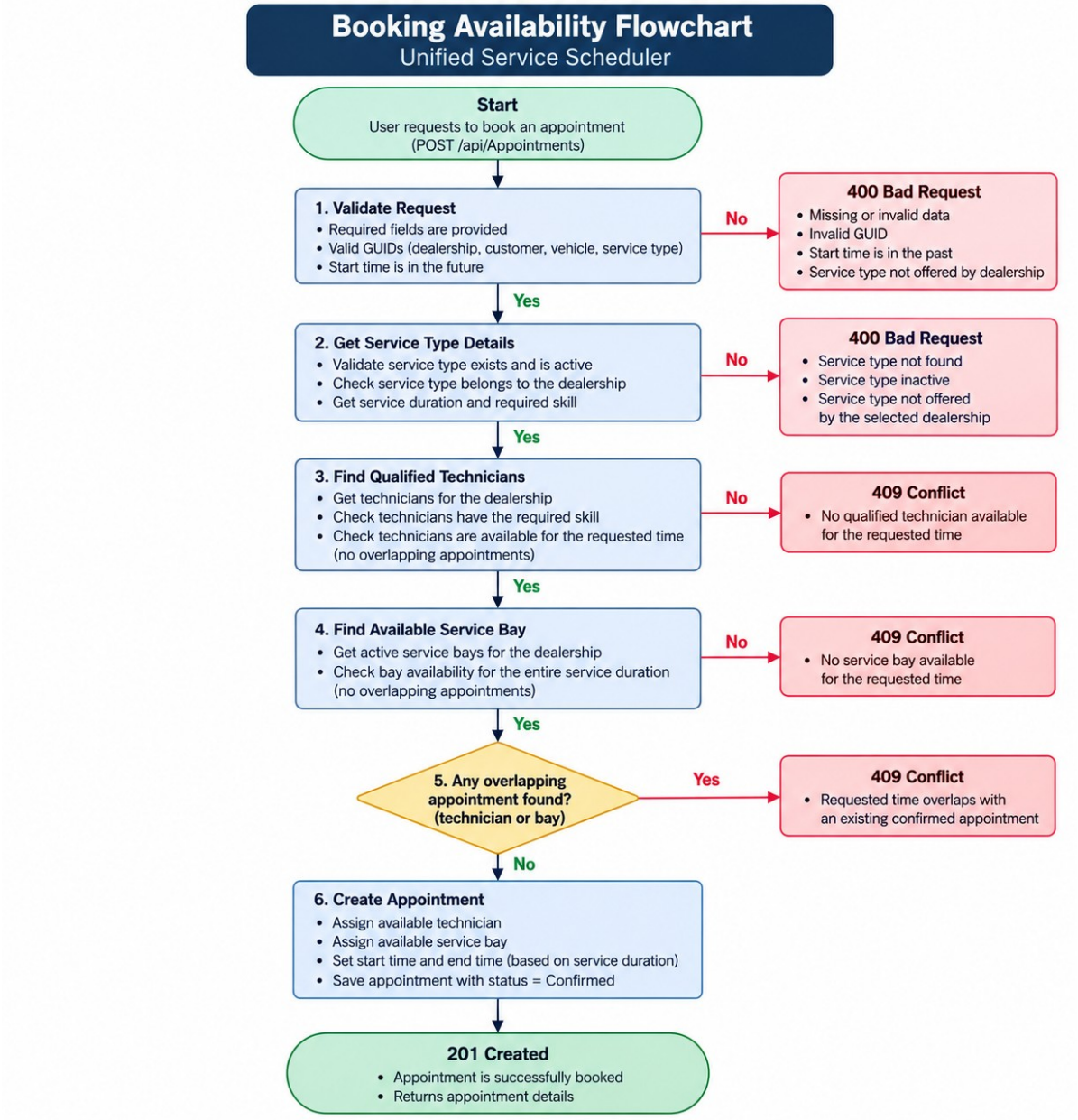


Figure 3. Decision path from request validation through resource assignment.

The flowchart is a conceptual view. In the implementation, technician availability and service-bay availability are evaluated separately. If either cannot be assigned, the booking service throws BookingConflictException and the controller maps it to HTTP 409.

6. Data Model and Persistence

6.1 Entity relationship diagram

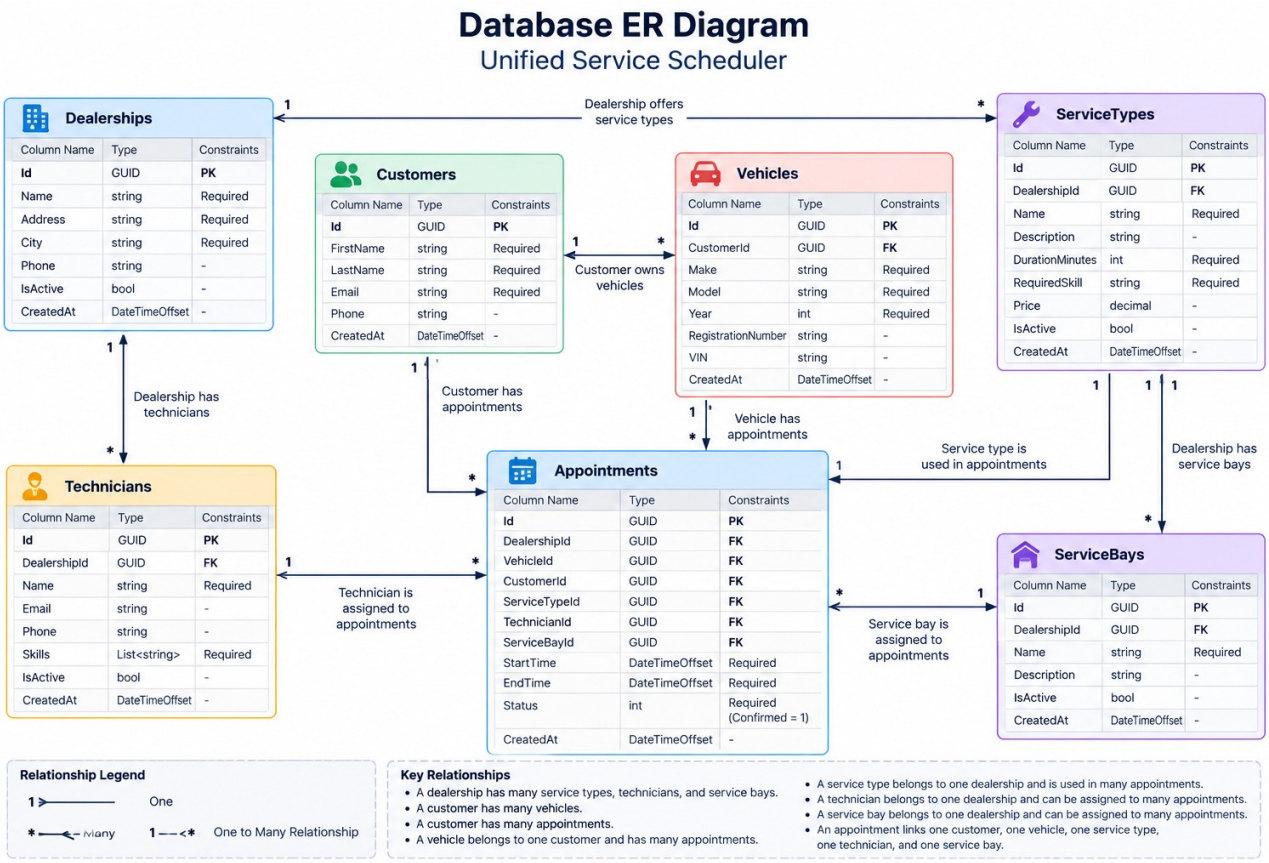


Figure 4. Logical entity relationships. Verify every field against the final entity classes before submission.

6.2 Core entities

Entity	Key data / purpose	Relationship role
Dealership	Id and dealership details	Parent of service types, technicians and service bays.
Customer	Id and customer details	Owns vehicles and requests appointments.
Vehicle	Id, CustomerId and vehicle details	Belongs to a customer and is associated with appointments.
ServiceType	Id, DealershipId, DurationMinutes, RequiredSkill	Defines the length and technician skill needed for a service.
Technician	Id, DealershipId, Skills, IsActive (if present)	Belongs to a dealership and can be assigned to appointments when qualified and free.
ServiceBay	Id, DealershipId, IsActive	Belongs to a dealership and can be assigned when active and free.
Appointment	Id, dealership/customer/vehicle/service/technician /bay IDs, StartTime, EndTime, Status	Records the confirmed booking and its assigned resources.

6.3 Persistence approach

- Entity Framework Core maps domain entities to relational tables and manages inserts/queries.
- Migrations version the schema and allow the database structure to be recreated consistently.
- The initial migration is part of the source repository; database initialization behavior should be described accurately based on Program.cs.
- SQLite is file-backed for the local demonstration, so appointments remain available after the API process restarts when the same database file is reused.
- A database transaction should cover any future multi-step writes that must succeed or fail together.

7. API Contract

7.1 Create appointment

Property	Type	Meaning
dealershipId	GUID	Dealership where the service will be performed.
customerId	GUID	Customer requesting the service.
vehicleId	GUID	Vehicle being serviced; must belong to the customer.
serviceTypeId	GUID	Service offered by the selected dealership.
startTime	ISO-8601 DateTimeOffset	Requested start time with an explicit offset or UTC designator.

```
POST                                                                    /api/Appointments
Content-Type:                                                           application/json

{
                                                    "dealershipId":     "<guid>",
                                                    "customerId":       "<guid>",
                                                    "vehicleId":        "<guid>",
                                                    "serviceTypeId":    "<guid>",
                                                    "startTime":        "2026-10-28T10:00:00+05:30"
}
```

Successful response: 201 Created, with appointment ID, assigned technician/bay IDs, start/end times and status.

7.2 Get appointment by ID

```
GET /api/Appointments/{id}
```

Returns 200 OK with the appointment when found; otherwise 404 Not Found.

7.3 HTTP status summary

Status	Meaning
201 Created	Appointment was saved; response includes the new appointment.

200 OK	Requested appointment exists.
400 Bad Request	Request data or referenced business data is invalid.
404 Not Found	Appointment ID was not found.
409 Conflict	No suitable technician or service bay is available for the requested interval.
500 Internal Server Error	Unexpected server-side failure.

8. Technology Choices

Technology	Why it was chosen	Trade-off / consideration
C# / .NET 8	Strong typing, async I/O, mature runtime and dependency injection.	Keep target framework and installed SDK aligned.
ASP.NET Core Web API	REST endpoints, model binding, middleware and Swagger/OpenAPI support.	Authentication/rate limiting would be needed for a public deployment.
Entity Framework Core 8	Typed queries, entity mapping, migrations and testable persistence.	Inspect generated SQL and query plans as data volume grows.
SQLite	Easy local setup and persistent file-based database for assessment.	Not the preferred choice for a highly concurrent, multi-instance production scheduler.
Swagger / OpenAPI	Interactive contract and quick manual verification.	Should be restricted or disabled appropriately in production.
xUnit	Automated tests for domain rules and booking scenarios.	Add more integration and concurrency tests as the application evolves.

9. Quality Attributes and Design Decisions

9.1 Maintainability

- Keep HTTP mapping in the controller and business rules in AppointmentBookingService.
- Use request/response DTOs instead of exposing EF entities directly.
- Keep domain entities in the Core project and infrastructure-specific behavior in the API project.
- Use descriptive exceptions for expected business outcomes; avoid using broad exception catches to conceal unrelated failures.

9.2 Performance

- Use asynchronous EF Core calls to avoid blocking request threads during database I/O.
- Use AsNoTracking for read-only queries.
- The current implementation loads confirmed appointments for the dealership and checks overlaps in memory. This is understandable for a small assessment dataset but grows linearly with appointment count.
- For a larger dataset, query only appointments whose intervals overlap the requested interval and whose technician/bay could affect the candidate assignment, using provider-supported time representation and appropriate indexes.

9.3 Reliability and concurrency

The current service uses a static SemaphoreSlim to serialize booking operations within a single API process. This reduces races between concurrent requests handled by that process, but it is not a distributed lock and does not coordinate across multiple application instances or processes. SQLite's locking behavior and a process-local semaphore alone are not a sufficient production guarantee against double booking.

Production hardening options:

- Use a database engine suited to concurrent production workloads, such as PostgreSQL or SQL Server.
- Perform resource selection and reservation within an explicit transaction at an appropriate isolation level.
- Introduce a durable reservation/slot model or database-enforced invariant where feasible; overlapping interval constraints require deliberate design and are not solved by a simple unique key.
- Retry only transient failures, and return a conflict when a competing transaction has already reserved the resource.
- Add integration tests that issue concurrent booking requests and verify that a technician or bay is never double-booked.

9.4 Scalability

The modular monolith is appropriate for this scope and can be scaled vertically for a demo. If traffic or team boundaries later justify horizontal scaling, first move to a shared production database and enforce booking invariants at that shared boundary. Only then consider multiple stateless API instances. Microservices, a message broker and distributed caching are not required for the current acceptance criteria.

10. Observability and Operations

The design should make failures diagnosable without leaking internal implementation details to clients. ASP.NET Core provides standard logging abstractions; verify that the application actually emits the expected logs before describing them as implemented.

Signal	What to capture	Purpose
Structured logs	Request/correlation ID, route, status code, elapsed time, exception details on the server.	Troubleshoot failed requests and latency.
Business events	Booking requested, validation rejected, resource conflict, booking confirmed.	Understand booking outcomes and conflict rates.
Metrics (production enhancement)	Request count, latency percentiles, 4xx/5xx rates, booking success/conflict counts, database failures.	Spot regressions and capacity issues.
Distributed tracing (production enhancement)	Trace controller → booking service → EF Core/database spans where instrumentation supports it.	Locate slow operations across dependencies.
Health checks (production enhancement)	API liveness and database readiness.	Support deployment monitoring and safe restarts.

Do not log sensitive customer data unnecessarily. Prefer stable identifiers and structured fields; never log credentials or secrets. For a production system, define dashboards and alert thresholds based on service-level objectives.

11. Testing Strategy and Evidence

11.1 Automated tests

The project has domain overlap tests and AppointmentBookingService tests. The earlier reported test run contained 14 passing tests. Re-run the suite against the final commit and record the actual count from the test runner.

Test area	Expected behavior
Overlap rule	Overlapping intervals conflict; adjacent intervals do not.
Successful booking	Persists an appointment with technician, bay, calculated end time and Confirmed status.
Technician qualification	Rejects booking if no technician has the required skill.
Technician availability	Rejects booking if all qualified technicians are occupied.
Bay availability	Rejects booking if no active bay is free.
Request validation	Rejects past/default times and invalid/missing references.
Customer/vehicle integrity	Rejects a vehicle that does not belong to the supplied customer.
Dealership/service integrity	Rejects a service type not associated with the selected dealership.
Adjacent booking	Allows a booking that starts exactly when the prior appointment ends.
API retrieval	Returns 200 for an existing appointment and 404 for an unknown ID.

11.2 Manual Swagger smoke tests

Scenario	Observed result
Valid POST /api/Appointments	201 Created — passed.
GET created appointment	200 OK — passed.
GET unknown appointment ID	404 Not Found — passed.
POST with past start time	400 Bad Request — passed.
POST same time with no alternative resources	409 Conflict — passed.
Restart API and GET existing appointment	200 OK — SQLite persistence passed.

These results are based on the developer-reported local test run. Include a screenshot or terminal output only if it is captured from the final code version.

12. Security Considerations

- The assessment prototype does not implement authentication or authorization. A real dealership system should authenticate users and enforce dealership-level access policies.
- Validate all request values server-side; never rely on a frontend to enforce booking rules.

- Use parameterized EF Core queries rather than constructing SQL from user input.
- Keep database files and secrets out of public source control unless the repository intentionally includes non-sensitive demo data.
- Return safe error messages to clients and keep detailed exception information in server-side logs.
- Apply HTTPS, rate limiting, request-size limits and appropriate Swagger exposure settings for a deployed service.

13. Assumptions and Open Questions

Question / ambiguity	Assumption used	Future clarification
How is technician qualification defined?	Technician Skills contains the service RequiredSkill, case-insensitively.	Should skills be normalized into a separate TechnicianSkill table?
Does service duration include cleanup/buffer time?	DurationMinutes is the full reserved interval.	Define per-service setup/cleanup buffers if required.
Can adjacent bookings be made?	Yes; intervals are half-open [start, end).	Confirm dealership operating hours and buffer rules.
What statuses consume resources?	Only Confirmed appointments block a technician and bay.	Define Pending, Cancelled, Completed and NoShow behavior if added.
Can the same vehicle have overlapping appointments?	Not explicitly prevented by the current implementation; technician and bay availability are the required constraints.	Usually a vehicle-level overlap rule should be considered and added if business policy requires it.
How should concurrent requests be handled across instances?	Single-process assessment protection only.	Select a database/transaction strategy before horizontal scaling.

14. GenAI Collaboration Narrative

Generative AI was used as an engineering collaborator to help structure the solution, draft implementation code, explain trade-offs, generate test cases and prepare the design artifacts. The engineer remained responsible for understanding the requirements, selecting the architecture and verifying the output.

14.1 How AI was directed

- The scenario was decomposed into explicit acceptance criteria: request validation, technician skill matching, service-bay availability, full-duration overlap checks, persistence and REST responses.
- The implementation was kept intentionally small: ASP.NET Core Web API, a booking service, EF Core, SQLite and xUnit rather than adding unnecessary infrastructure.
- AI assistance was requested in small, reviewable steps, including domain rules, service implementation, controller behavior and tests.
- The prompts were constrained by the assessment acceptance criteria and the existing project structure; changes were requested as focused, replaceable files instead of broad rewrites wherever possible.
- The AI was asked to explain edge cases explicitly, including half-open time intervals, adjacent appointments, service duration, technician skill matching, and the distinction between validation failures and resource conflicts.
- The design was kept proportionate to the assessment: a modular monolith was preferred over premature microservices, and proposed production improvements were documented separately from features actually implemented.

14.2 Verification and refinement

- The solution was built and the automated test suite was run; the reported test result was 14 passing tests.
- Swagger was used to exercise a successful create request and inspect the returned appointment/resource IDs.
- Manual tests verified GET 200, unknown ID 404, past-time validation 400, resource conflict 409 and persistence after restarting the API.
- The overlap rule and adjacent-time behavior were explicitly tested.
- The concurrency limitation of a process-local SemaphoreSlim was identified and documented rather than claiming it solves multi-instance booking races.
- Generated code and diagrams must be compared against the actual repository before submission; AI output is treated as a proposal to review, not as authoritative evidence.

14.3 Ownership statement

I take ownership of the final solution by understanding the booking flow, being able to explain the validation and overlap rules, rerunning tests on the submitted revision, and clearly distinguishing what is implemented today from production improvements. I would not claim features such as distributed locking, authentication, metrics dashboards or distributed tracing unless they are actually present in the code.

15. Production Evolution Roadmap

Priority	Improvement	Reason
P0	Database-backed concurrency control and concurrent integration tests	Prevent double-booking across processes/instances.
P0	Verify and standardize exception handling	Avoid mapping unrelated internal errors to misleading client responses.
P1	Add cancellation/rescheduling with transactional resource release	Support normal appointment lifecycle.
P1	Add authentication and dealership-scoped authorization	Protect customer and dealership data.
P1	Add operating hours, time-zone policy and booking horizon validation	Reflect real dealership availability.
P2	Optimize overlap queries and add appropriate indexes	Avoid loading all confirmed appointments as data volume grows.
P2	Add structured business metrics, tracing and health checks	Improve production diagnosis and operations.
P3	Evaluate reminders, notifications and async workflows	Support customer communications without slowing booking requests.

16. Conclusion

The current implementation demonstrates the core scheduling use case: validate the request, determine service duration and required skill, check technician and bay availability for the full interval, persist the assigned appointment, and expose REST endpoints with meaningful status codes. The design is intentionally simple and testable for the assessment. Its most important known limitation is that the

process-local lock does not provide distributed concurrency safety. The next engineering step for production would be to enforce booking invariants transactionally at a shared database boundary and prove the behavior with concurrent integration tests.